

IAAA Competition 2026: Brain CT Triage Challenge

Brain CT Triage Challenge

Overview

In this challenge, you will develop models that perform **clinically meaningful triage** on non-contrast brain CT scans.

Instead of directly predicting triage classes, your model will predict a set of **intermediate imaging quantities** for each CT study:

1. **Total volume** (in mL) of each intracranial hemorrhage (ICH) subtype.
2. **Probability of any skull fracture**.
3. **Midline shift (MLS)** in mm.

A **fixed, clinically motivated rule** (provided as executable Python code) then converts these intermediate values into **triage classes**:

- **0 – Non-urgent**
- **1 – Urgent**
- **2 – Critical**

The **primary evaluation metric** is **Quadratic Weighted Kappa (QWK)** between your model's triage output and the ground-truth triage. In the event of a **tie in QWK**, we use the **Mean Absolute Error (MAE)** of total ICH volume as a **tie breaker**.

Participants will **not** submit CSV prediction files. Instead, you will submit:

- Your **model files** (weights, configs, etc.), and
- A **Python module** implementing a standard **Model API**.

Our evaluation pipeline will load your code and models and run inference directly on the hidden evaluation set.

Problem Description

Non-contrast brain CT is a first-line imaging modality in emergency neuroradiology. Rapid triage is crucial to identify patients who:

- Need **immediate neurosurgical or neurocritical care** (critical),
- Require **urgent but not emergent** attention (urgent),
- Have **no acute, triage-relevant findings** (non-urgent).

Clinically, triage is primarily driven by a small set of **imaging primitives**:

- **Intracranial hemorrhage burden** (how much blood, and of what type?),
- **Presence of skull fracture**,
- **Degree of midline shift / mass effect**.

You will build models that predict these primitives from CT images. A transparent, fixed triage rule converts those primitives to triage classes. This design ensures:

- **Interpretability** – your model predicts clinically meaningful quantities.
- **Mechanistic behavior** – triage is derived from explicit rules.
- **Fairness / reproducibility** – the same rule is used for both ground truth and predictions.

Labels and Tasks

Available annotations (training set)

For each CT **study** in the training set, we provide:

- **Intracranial hemorrhage (ICH) segmentation masks** Five semantic classes (exact names defined in the Data Profile), typically including:
 - Epidural hemorrhage (**EDH**)
 - Subdural hemorrhage (**SDH**)
 - Intraparenchymal / intracerebral hemorrhage (**IPH/ICH**)
 - Subarachnoid hemorrhage (**SAH**)
 - Intraventricular hemorrhage (**IVH**)

There may be **multiple lesions per ICH subtype**. All instances of a given subtype are included in its mask. The **volume per subtype is defined as the combined volume of all lesions** of that subtype in the study.

- **Skull fractures** 2D bounding boxes indicating the locations of skull fractures on each annotated slice.
- **Midline shift keypoints** For slices where midline annotations are provided, three keypoints are stored:
 - "AnteriorFalxAttachment"
 - "PosteriorFalxAttachment"
 - "OutermostPointOfTheFalx"

These keypoints can be used to reconstruct the **midline** and compute slice-wise midline deviation. For the challenge, ground-truth **midline shift at the foramen of Monro level** (`MLS_mm`) is computed from these keypoints and provided as part of the **intermediate ground truth** for each series.

These annotations are provided for **training** (and optionally a labeled validation subset). The **test set** includes only the CT images (no segmentation, no keypoints, no fracture boxes).

Data Profile & Dataset Structure

The dataset consists of three main components:

1. **Metadata table** (e.g., CSV or Parquet)
2. **Raw DICOM data**
3. **Annotation JSON files**

1. Metadata

A metadata file is provided (for example `metadata.csv` or `metadata.parquet`) that contains:

- **DICOM-level information** (e.g., `dicom_series.id` , number of slices, acquisition parameters, etc.).
- **Aggregated labels** derived from per-slice annotations and keypoints, including:
 - Series-level ICH volumes per subtype:
 - `V_EDH` , `V_SDH` , `V_IPH` , `V_SAH` , `V_IVH` (mL)
 - Series-level midline shift:
 - `MLS_mm` (mm at the foramen of Monro level)
 - Series-level fracture presence:
 - `SkullFracture` (boolean) or fracture count, which maps to `fracture_prob` = 0 or 1 in the intermediate schema.
 - Optional triage labels for the training set (for debugging).

This metadata is primarily for **training** and quality control; the **official triage labels** are always derived via `triage_from_intermediates` from the intermediate primitives.

2. Raw DICOM data

All raw images are provided as DICOM files under a root directory:

```
dicom/  
  {dicom_series.id}/  
    *.dcm
```

For each series:

- `dicom/{dicom_series.id}/` contains all DICOM slices belonging to that series.
- Your model's `predict(study_dir)` method will receive the path:
 - `study_dir = "dicom/{dicom_series.id}"`

You are free to use any DICOM loading strategy that works inside the official environment (e.g., `pydicom`, `SimpleITK`, MONAI loaders).

3. Annotation JSON files

Per-slice annotations (segmentation, keypoints, fracture boxes) are stored as JSON files:

```
annotations/
  {dicom_series.id}/
    {SOPInstanceUID}.json
```

Each JSON file for a given DICOM instance (`SOPInstanceUID`) contains:

- A **segmentation map** encoded as a 2D label mask (ICH subtypes encoded as integer labels) stored as RLE (Run-Length Encoding).
- A **class map** describing the mapping from integer label → class name.
- A **keypoints** dictionary mapping keypoint name → (x, y) coordinates (or `None`).
- A list of **boxes** in `[x, y, w, h]` format, typically used for skull fractures.

Intermediate Quantities & Triage Rules

Intermediate schema (per series)

For each CT series, your model must output the following intermediate values:

- `"V_EDH"` – total epidural hemorrhage volume in mL
- `"V_SDH"` – total subdural hemorrhage volume in mL
- `"V_IPH"` – total intraparenchymal / intracerebral hemorrhage volume in mL
- `"V_SAH"` – total subarachnoid hemorrhage volume in mL
- `"V_IVH"` – total intraventricular hemorrhage volume in mL
- `"fracture_prob"` – probability (0–1) of any skull fracture in the series
- `"MLS_mm"` – midline shift at the foramen of Monro level in mm

The **same schema** is used for both:

- Ground truth intermediates: `y_true[series_id]`
- Predicted intermediates: `y_pred[series_id]`

Triage classes

- 0 – Non-urgent
- 1 – Urgent
- 2 – Critical

Official triage function

This is the **canonical triage function** that will be used to derive triage labels from intermediate values:

```
from typing import Any, Dict, Mapping

\
TRIAGE_REQUIRED_KEYS = {
    "V_EDH",
    "V_SDH",
    "V_IPH",
    "V_SAH",
    "V_IVH",
    "fracture_prob",
    "MLS_mm",
}

\
def validate_intermediates(intermediates: Mapping[str, Any]) -> Dict[str, float]:
    """Validate and normalize intermediate values for a single series.

    This helper checks that a mapping (for example, one entry in y_true or
    y_pred) contains exactly the keys expected by the triage function and
    that all values are convertible to Python floats.

    Expected keys for each series:

        * "V_EDH": total epidural hemorrhage volume in mL
        * "V_SDH": total subdural hemorrhage volume in mL
        * "V_IPH": total intraparenchymal/intracerebral hemorrhage volume
          in mL
        * "V_SAH": total subarachnoid hemorrhage volume in mL
        * "V_IVH": total intraventricular hemorrhage volume in mL
        * "fracture_prob": probability (0-1) of any skull fracture
        * "MLS_mm": midline shift at the foramen of Monroe level in mm

    The same schema is used for both y_true and y_pred, even though they
    represent ground truth and predictions, respectively.

    Args:
        intermediates: Mapping with intermediate values for a single series.

    Returns:
        A new dictionary mapping each required key to a float.
```

Raises:

ValueError: If required keys are missing or unexpected keys are present.

TypeError: If any value cannot be converted to float.

"""

missing = TRIAGE_REQUIRED_KEYS - intermediates.keys()

extra = intermediates.keys() - TRIAGE_REQUIRED_KEYS

if missing:

```
    raise ValueError(
        f"Missing keys in intermediates: {sorted(missing)}. "
        f"Expected keys: {sorted(TRIAGE_REQUIRED_KEYS)}."
    )
```

if extra:

```
    raise ValueError(
        f"Unexpected keys in intermediates: {sorted(extra)}. "
        f"Expected keys: {sorted(TRIAGE_REQUIRED_KEYS)}."
    )
```

cleaned: Dict[str, float] = {}

for key in TRIAGE_REQUIRED_KEYS:

```
    try:
        cleaned[key] = float(intermediates[key])
    except Exception as exc: # noqa: BLE001
        raise TypeError(
            f"Value for key {key!r} must be convertible to float, "
            f"got type {type(intermediates[key]).__name__}."
        ) from exc
```

return cleaned

\

def triage_from_intermediates(intermediates: Mapping[str, Any]) -> int:

"""Compute triage class from intermediate imaging primitives.

This function implements a fixed, clinically inspired set of rules that convert intermediate quantities for a single CT study into a triage label:

- * 0 = non-urgent
- * 1 = urgent
- * 2 = critical

The same expected keys and logic are used for both ground truth intermediates (y_true) and predicted intermediates (y_pred). All thresholds are hard-coded in this function so that post-processing is fully deterministic and consistent across users.

Expected keys in `intermediates`:

- * "V_EDH": total epidural hemorrhage volume in mL (≥ 0)
- * "V_SDH": total subdural hemorrhage volume in mL (≥ 0)
- * "V_IPH": total intraparenchymal/intracerebral hemorrhage volume

```

    in mL ( $\geq 0$ )
* "V_SAH": total subarachnoid hemorrhage volume in mL ( $\geq 0$ )
* "V_IVH": total intraventricular hemorrhage volume in mL ( $\geq 0$ )
* "fracture_prob": probability (0-1) that any skull fracture is
    present
* "MLS_mm": midline shift at the foramen of Monro level in mm ( $\geq 0$ )

```

Core behavior:

```

* Small hemorrhage volumes ( $< 0.1$  mL) and very small MLS ( $< 1$  mm)
  are treated as noise.
* High MLS is considered critical only if there is intracranial
  hemorrhage or fracture. If MLS is high with neither hemorrhage nor
  fracture, the case is considered urgent.
* ICH-based critical thresholds are relatively high to avoid an
  excessively large critical class and bias. Moderate hemorrhages
  without major MLS are typically urgent rather than critical.

```

Args:

```

intermediates: Mapping with intermediate values for one CT series.
    The mapping may be one entry from y_true or y_pred.

```

Returns:

```

Triage class as an integer:
* 0 = non-urgent
* 1 = urgent
* 2 = critical

```

Raises:

```

ValueError: If required keys are missing or unexpected keys are
    present.
TypeError: If any intermediate value cannot be converted to float.

```

```

"""

```

```

# Normalize and validate keys / types
vals = validate_intermediates(intermediates)

```

```

# Extract and clamp primitives
V_EDH = max(0.0, vals["V_EDH"])
V_SDH = max(0.0, vals["V_SDH"])
V_IPH = max(0.0, vals["V_IPH"])
V_SAH = max(0.0, vals["V_SAH"])
V_IVH = max(0.0, vals["V_IVH"])
MLS_mm = max(0.0, vals["MLS_mm"])
fracture_prob = float(vals["fracture_prob"])

```

```

total_vol = V_EDH + V_SDH + V_IPH + V_SAH + V_IVH

```

```

# -----
# Hard-coded thresholds
# -----

```

```

EPS_VOLUME = 0.1      # mL: ignore tiny volumes as noise
EPS_MLS = 1.0         # mm: ignore <1 mm as no meaningful shift

```

```

MLS_CRITICAL = 5.0      # mm: high MLS threshold
MLS_URGENT_LOW = 3.0    # mm: moderate MLS range lower bound

EDH_CRIT = 30.0         # mL
SDH_CRIT = 70.0         # mL
IPH_CRIT = 70.0         # mL
TOTAL_VOL_CRIT = 60.0   # mL

COMBO_MLS = 3.0         # mm: for MLS + volume combo rule
COMBO_VOL = 40.0        # mL

FRAC_VOL_CRIT = 15.0    # mL: fracture + hemorrhage for critical

FRACTURE_PRESENCE_THRESHOLD = 0.5 # fracture probability threshold

# Derived flags
has_ich = total_vol >= EPS_VOLUME
mls_present = MLS_mm >= EPS_MLS
fracture_present = fracture_prob >= FRACTURE_PRESENCE_THRESHOLD

# -----
# Critical triage (2)
# -----

# 1) MLS-driven critical *only if* there is ICH or fracture
if MLS_mm >= MLS_CRITICAL and (has_ich or fracture_present):
    return 2

# 2) Large single-compartment hemorrhages
if V_EDH >= EDH_CRIT:
    return 2
if V_SDH >= SDH_CRIT:
    return 2
if V_IPH >= IPH_CRIT:
    return 2

# 3) Large total hemorrhage burden
if total_vol >= TOTAL_VOL_CRIT:
    return 2

# 4) Combined hemorrhage + MLS rule
if has_ich and MLS_mm >= COMBO_MLS and total_vol >= COMBO_VOL:
    return 2

# 5) Fracture with substantial hemorrhage
if fracture_present and total_vol >= FRAC_VOL_CRIT:
    return 2

# -----
# Urgent triage (1)
# -----

# 6) MLS in critical range but no hemorrhage and no fracture

```



```

if MLS_mm >= MLS_CRITICAL and not (has_ich or fracture_present):
    return 1

# 7) Any meaningful hemorrhage
if has_ich:
    return 1

# 8) Moderate MLS range
if MLS_URGENT_LOW <= MLS_mm < MLS_CRITICAL:
    return 1

# 9) Fracture with small hemorrhage
if fracture_present and total_vol < FRAC_VOL_CRIT:
    return 1

# 10) Small hemorrhage with any non-trivial MLS
if total_vol >= EPS_VOLUME and mls_present:
    return 1

# -----
# Non-urgent triage (0)
# -----
return 0

```

Model API & Environment

Evaluation environment

All models will be evaluated in a controlled environment with:

- **Hardware**
 - 1 × GPU with **24 GB VRAM** (e.g., NVIDIA RTX-class GPU)
 - Sufficient CPU and RAM for typical medical imaging workloads
- **Software**
 - A predefined **Docker image** containing:
 - Python (version to be specified, e.g., 3.10)
 - PyTorch, TensorFlow/Keras, and commonly used libraries
 - `pydicom` and supporting packages
 - The exact package versions and Docker image name will be specified on the competition page.

Important:

- Your code **must** run in this environment **without installing extra packages at evaluation time**.
- Your model must complete inference on the hidden evaluation set within a **global time limit** on a single 24 GB GPU (target ≈ 30 minutes; design for ≤ 15 minutes).

- All **prediction logic**, including preprocessing, batching, and ensembling, is implemented inside your code.

Rules

- **Data usage**
 - Only use the **official competition data** and any external resources explicitly allowed by the organizers.
 - Do not attempt to re-identify patients; all data are anonymized.
- **Annotations**
 - You may fully use the provided segmentation masks, fracture boxes, and keypoints for training.
 - You must not manually label or otherwise access ground truth on the hidden test set.
- **Model submission**
 - Your submission must contain `model.py` and a `models/` directory with all necessary weights.
 - Your model must run in the official Docker image and satisfy the time limit on a single 24 GB GPU.
- **Ensembling**
 - Ensembling multiple models is allowed, as long as the full inference pipeline completes within time and memory limits.
- **Reproducibility**
 - Top teams may be required to share additional details (training code, configs, etc.) to verify results.
- **Fair play**
 - Sharing of general ideas and open-source code is encouraged.
 - Sharing of private test predictions, test labels, or attempts to reverse-engineer the evaluation system is strictly prohibited.
 - Organizers reserve the right to disqualify teams for non-compliance or suspicious behavior.